SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	**ABDUL WAHID. H**	**Reg. No.:**	**192321079**
Section / Batch:	**B.TECH IT**	**Date:**	**03/08/2026**

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct

*I **Abdul Wahid. H / 192321079** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ABDUL WAHID. H

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Given

Add +120 and +50 using 8-bit signed 2's complement.

An 8-bit signed number has the range:

−128 to +127-128 \text{ to } +127−128 to +127

Step 1: Convert to binary

Decimal	**8-bit Binary**
+120	01111000
+50	00110010

Step 2: Binary addition

01111000 (+120)

+ 00110010 (+50)

10101010

Mathematically,

120+50=170120+50=170120+50=170

But 170 cannot be represented in signed 8-bit format, whose maximum positive value is +127.

The resulting bit pattern 10101010, if interpreted as signed 2's complement, represents −86, not +170.

Debugging the issue

The problem is signed overflow.

For 2's complement addition, overflow occurs when:

- Two positive numbers produce a negative result, or
- Two negative numbers produce a positive result.

Here:

120 → sign bit = 0

50 → sign bit = 0 Result → sign bit

= 1

Therefore:

Overflow occurred\boxed{\text{Overflow occurred}}Overflow occurred Overflow can also

be detected using:

V=Cin to MSB⊕Cout of MSBV=C_{in\ to\ MSB}\oplus C_{out\ of\ MSB}V=Cin to MSB⊕Cout of MSB

Solution

The processor should check the overflow flag (V) after signed addition.

Alternatively, use a larger representation such as 16 bits:

00000000 01111000 (+120)

00000000 00110010 (+50)

00000000 10101010 (+170)

Conclusion

The incorrect result is caused by exceeding the signed 8-bit range. Overflow detection or a wider data type should be used to handle the result correctly.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Ripple Carry Adder

In a Ripple Carry Adder (RCA), the carry generated by one full-adder stage must propagate to the **next stage** before the final result can be determined.

For a 4-bit RCA:

A0,B0 → FA0 → C1

↓

A1,B1 → FA1 → C2

↓

A2,B2 → FA2 → C3

↓

A3,B3 → FA3 → C4

Debugging the performance bottleneck Consider:

1111

+ 0001

10000

Carry propagation occurs as follows:

Stage Operation	**Carry**	
Bit 0	1 + 1	C1 = 1
Bit 1	1 + 0 + C1	C2 = 1
Bit 2	1 + 0 + C2	C3 = 1
Bit 3	1 + 0 + C3	C4 = 1

Every stage waits for the preceding carry.

For an **n-bit RCA**, worst-case carry delay grows approximately linearly:

TRCA∝nT_{RCA}\propto nTRCA∝n

This becomes a performance bottleneck in real-time systems.

Optimized Solution: Carry Look-Ahead Adder

A Carry Look-Ahead Adder (CLA) calculates carries using **Generate (G)** and **Propagate (P)** signals.

For each bit:

Gi=AiBiG_i=A_iB_iGi=AiBi Pi=Ai⊕BiP_i=A_i\oplus B_iPi=Ai⊕Bi Carry equation:

Ci+1=Gi+PiCiC_{i+1}=G_i+P_iC_iCi+1=Gi+PiCi Therefore:

C1=G0+P0C0C_1=G_0+P_0C_0C1=G0+P0C0

C2=G1+P1G0+P1P0C0C_2=G_1+P_1G_0+P_1P_0C_0C2=G1+P1G0+P1P0C0

and similarly for higher bits.

RCA vs CLA

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry calculation	Sequential	Parallel/look-ahead
Delay	High	Low
Hardware	Simple	More complex
Speed	Slower	Faster
Real-time systems	Less suitable	More suitable

Conclusion

The RCA is slow because the carry ripples sequentially through all stages. A CLA reduces the delay by computing carry signals from generate/propagate logic instead of waiting for each previous full adder.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Booth's algorithm performs efficient signed binary multiplication using 2's complement numbers.
It examines the pair:
(Q0,Q−1)(Q_0,Q_{-1})(Q0,Q−1)

Booth's Rules

Q0Q−1Q_0Q {-1}Q0 Q−1	**Operation**
00	No operation
01	A=A+MA=A+MA =A+M
10	A=A−MA=A- MA=A−M
11	No operation

After each operation, perform an arithmetic right shift on:
[A,Q,Q−1][A,Q,Q_{-1}][A,Q,Q−1]

Example: (−3)×2(-3)\times 2(−3)×2 Using 4-bit
Representation:
−3=1101-3=1101−3=1101 2=00102=00102=0010

Initialize:

M = 1101 (-3)

A = 0000

Q = 0010 (+2)

Q-1 = 0

Step-by-step concept At each iteration:

Check Q0 Q-1.

1. Perform add/subtract/no operation.
2. Arithmetic-right-shift A,Q,Q-1.
3. Preserve the sign bit.
4. Repeat for the number of operand bits.

The expected final decimal result is:

(−3)(2)=−6(-3)(2)=-6(−3)(2)=−6 8-bit representation:

11111010

Possible sources of error

1. Incorrect bit-pair checking For example, implementing:

01 → A - M 10 → A + M

would reverse Booth's required operations.

Correct:

01 → A + M

10 → A - M

2. **Logical shift instead of arithmetic shift** A logical right shift inserts 0 into the MSB.

That is incorrect for negative signed numbers.

An arithmetic right shift must copy the previous sign bit.

Example:

Before: 1101

Arithmetic shift: 1110

Logical shift: 0110 ← incorrect for signed value

3. **Incorrect sign extension**

Negative operands must retain their sign when the bit width is extended.

Example:

4-bit -3 = 1101 8-bit -3 =

11111101 Not:

00001101

4. **Wrong number of iterations**

For an n-bit multiplier, Booth's algorithm should execute n iterations.

Corrections

- Correctly implement 01 and 10 rules.
- Use arithmetic right shifts.
- Preserve the sign bit.
- Apply proper sign extension.
- Use fixed-width 2's complement registers.
- Execute exactly n iterations.

Conclusion

Incorrect results for negative operands usually arise from incorrect arithmetic shifts, sign extension, or reversed Booth bit-pair operations. Correct signed handling ensures accurate multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Restoring Division

Restoring division repeatedly subtracts the divisor from the partial remainder.

If the result becomes negative, the divisor must be **added back**, or "restored."

Basic workflow

1. Shift A and Q left
2. A = A - M
3. Check A
4. If A < 0:

   Q0 = 0

   A = A + M ← Restoration

   Else:

   Q0 = 1
5. Repeat Where:
   - A = partial remainder
   - Q = dividend/quotient
   - M = divisor

Debugging the inefficiency Suppose:

A−M<0A-M<0A−M<0

The processor first performs:

A=A−MA=A-MA=A−M

and then immediately performs:

A=A+MA=A+MA=A+M

to restore the old value.

Therefore, two arithmetic operations may be performed merely to return A to its previous state.

Main bottleneck

Subtract divisor

↓

Negative?

↙ ↘

Yes No

↓ ↓

Restore Continue

A=A+M

Repeated restorations increase execution time.

Optimized Approach: Non-Restoring Division

Non-restoring division avoids immediately restoring a negative partial remainder.

Instead, the next iteration chooses addition or subtraction based on the sign of the previous remainder.

Basic rule

If:

A≥0A\geq0A≥0

perform:

A=A−MA=A-MA=A−M

If:

A<0A<0A<0

perform:

A=A+MA=A+MA=A+M

Thus, the algorithm does not perform an immediate restoration after every negative subtraction.

Comparison

Feature	Restoring	Non-Restoring
Negative remainder	Restored immediately	Used in next iteration
Extra addition	Frequent	Reduced
Arithmetic operations	More	Fewer
Execution time	Higher	Lower
Performance	Lower	Better

Why performance improves

Suppose restoring division generates:

A−M<0A-M<0A−M<0

It performs:

SUB M

ADD M

Non-restoring division can continue without the immediate ADD M, using the sign of A to determine the next operation.

Therefore, unnecessary restoration operations are eliminated.

Conclusion

The main inefficiency in restoring division comes from repeated restoration of negative partial remainders. Non-restoring division improves performance by postponing correction and reducing redundant arithmetic operations.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

IEEE 754 **single precision** uses 32 bits:

Sign	**Exponent**	**Fraction**
1 bit	8 bits	23 bits

The effective precision is approximately **24 binary significant bits** because normalized values have an implicit leading 1.

Problem

Consider adding a very large number and a very small number:

1.0×108+1.01.0\times10^8+1.01.0×108+1.0 Mathematically:

100000000+1=100000001100000000+1=100000001100000000+1=100000001

But in single-precision floating point, the result may effectively remain: 10000000010000000100000000

because the smaller contribution cannot always be represented at that magnitude.

Step 1: Exponent alignment

Before floating-point numbers are added, their exponents must be made equal.

Conceptually:

1.0×2E11.0\times2^{E_1}1.0×2E1 and

1.0×2E21.0\times2^{E_2}1.0×2E2

If:

E1≫E2E_1 \gg E_2E1≫E2

the significand of the smaller number must be shifted right many positions.

Large number: 1.xxxxxxxxx × 2^E

Small number: 0.000000...1 × 2^E

The significant bits of the smaller value may move beyond the available precision.

Step 2: Loss of precision

Single precision provides only a limited number of significant bits.

When the smaller number's bits are shifted beyond this precision, they are discarded or only influence rounding.

Thus:

Large+Small≈LargeLarge+Small\approx LargeLarge+Small≈Large

This is a **rounding/precision limitation**, not necessarily a processor bug.

Step 3: Normalization

After significands are added, the result is normalized into the form:

1.f×2E1.f\times2^E1.f×2E

If normalization requires shifting, the exponent is adjusted accordingly.

Additional low-order bits can again be lost during this process.

Step 4: Rounding

IEEE 754 typically uses **round to nearest, ties to even** by default.

Bits beyond the representable significand affect rounding, but sufficiently tiny contributions may have no effect on the final stored number.

Optimization Techniques

1. **Use double precision**

IEEE 754 double precision uses:

Field	Bits
Sign	1
Exponent	11
Fraction	52

It has an effective precision of about **53 binary significant bits**.
Therefore:
Single precision → ~7 decimal significant digits

Double precision → ~15–16 decimal significant digits Double precision
greatly reduces rounding errors.

2. **Change the order of operations**

When summing many values, adding values of similar magnitude or adding **smaller values first** can reduce loss of precision.

Instead of:
Large+Small1+Small2+…Large+Small_1+Small_2+\cdotsLarge+Small1+Small2+…
accumulate smaller terms first where practical.

3. **Kahan Summation**

For long sequences of floating-point additions, **Kahan compensated summation** keeps track of some lost low-order information.

It can substantially reduce accumulated rounding error.

4. **Scaling**

Where appropriate, values can be rescaled so that operands have more similar magnitudes before arithmetic operations.